A LOW-CODE, NO-CODE
INFORMATICS PLATFORM

Sapio Implementing a Unit Testing Framework Guide (JUnit 5, Maven, BDD & Cucumber)

Developing reliable software requires a solid testing strategy. This guide outlines how to establish a comprehensive unit testing framework within the Sapio platform, leveraging Jupiter JUnit, the AbstractExemplarTester, Maven, and Behavior-Driven Development (BDD) principles with Cucumber. This approach aims to capture requirements effectively, inspire confidence in releases, reduce development costs through automation, and improve overall code quality.

Note : We use Java plugins as prime examples of discrete units that can be tested in this doc, but unit testing can be designed and applied for any piece of code that can be isolated.

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620

This document is **RESTRICTED**

Table of Contents

What is Unit Testing in Sapio? 3
What frameworks do we use? 3
Justification for the Approach 3
- Traditional Testing Challenges 3
- The Sapio Testing Procedure & Mission 4

Toolkit Requirements 4
Getting Started with Unit Testing in Sapio 4
- IDE Setup for Unit Testing (IntelliJ JUnit Plugin) 4
- Project Setup for Unit Testing 6
- Unit Test Class Setup (extending AbstractExemplarTester) 7

Mocking Sapio Unit Test Resources 8
- Datatypes and Datafields (Synergy Files) 8
- Creating a Mock Experiment 10
- Utilizing additionalBeforeEach() 11

Writing and Running JUnit Tests 11
- Basic Test Structure (already shown in Unit Test Class Setup) 11
- Running Tests from Project Panel, Class, or Method (IntelliJ) 11
  - From Project Panel (Directory or Class) 11
  - From a Class or Method in Editor 12
- Debugging JUnit Tests 13
- Viewing Test Results in IntelliJ 13

Measuring Test Coverage 14
Integrating with Maven 14
Behavior-Driven Development (BDD) with Cucumber 15
- Why BDD? 15
- The BDD Workflow 15
- Introducing Cucumber 16
  - Cucumber is a BDD tool using three main file types 16
  - Location of Cucumber Files 16
  - IDE Plugin for Cucumber 16
  - Crafting Feature Files (.feature) 16
  - Implementing Step Definitions (.java) - Leveraging AbstractExemplarTester 17
  - Using a Test Described in a Table for Batch Conditions 18
  - Setting Up the Cucumber Test Runner (.java) 21
  - Debugging Cucumber Tests 21

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620

This document is **RESTRICTED**

Automated Reporting and CI/CD 22
Generating Reports with Maven 22
Visualizing Results in Jenkins (or other CI servers) 22
Best Practices & Tricks of the Trade 22
Conclusion 23

What is Unit Testing in Sapio?

Unit testing consists of a set of Java classes containing test methods that typically conclude with some sort of "assert" statement. The purpose of a unit test is to run a method or plugin with a predefined input that should produce a predictable output. Developing unit tests as part of plugin development can help to ensure that new changes don't have unintended consequences on previous plugins.

What frameworks do we use?

Sapio uses the Jupiter JUnit (JUnit 5) testing framework. The Jupiter JUnit framework contains the various "assert" methods that you can call that will be used to determine pass/fail status on your tests.

WARNING : Be sure that you are using JUnit 5, as the function described below depends on this version

Justification for the Approach

Traditional Testing Challenges

Traditional testing often involves late-cycle integration tests, UAT close to production, developers solely responsible for initial tests, and extensive re-testing. This can lead to...

- **Inaccuracy:** Forgetting original intent over time, leading to regressions.
- **Expansiveness:** Time-consuming re-reading of documents or code.
- **Ambiguity:** Misunderstandings persisting until late-stage testing.

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

The Sapio Testing Procedure & Mission

Our mission is to address these by...

1. **Capturing requirements** during testing using Behavior-Driven Development (BDD)
2. **Inspiring confidence** by linking tests to the build system, mandating passed tests, and auto-generating reports
3. **Reducing costs** by automating regression testing and creating living documentation
4. Enabling developers to perform **functional unit tests and debug directly within their IDE**

Toolkit Requirements

Developers wishing to perform unit testing are recommended to have the following...

- **Jupiter JUnit:** The primary Java testing framework used for writing test methods and assertions
- **AbstractExemplarTester (Sapio Commons Test Driver):** A base class provided by Sapio to simulate the server environment, mock dataMgmtServer, user, dataRecordManager, and manage test context (like loading datatypes)
- **Maven:** The build automation tool used to compile code, manage dependencies, and execute tests
- **Cucumber:** A tool for Behavior-Driven Development, allowing tests to be written in a natural language format (Gherkin) and then linked to Java code

Getting Started with Unit Testing in Sapio

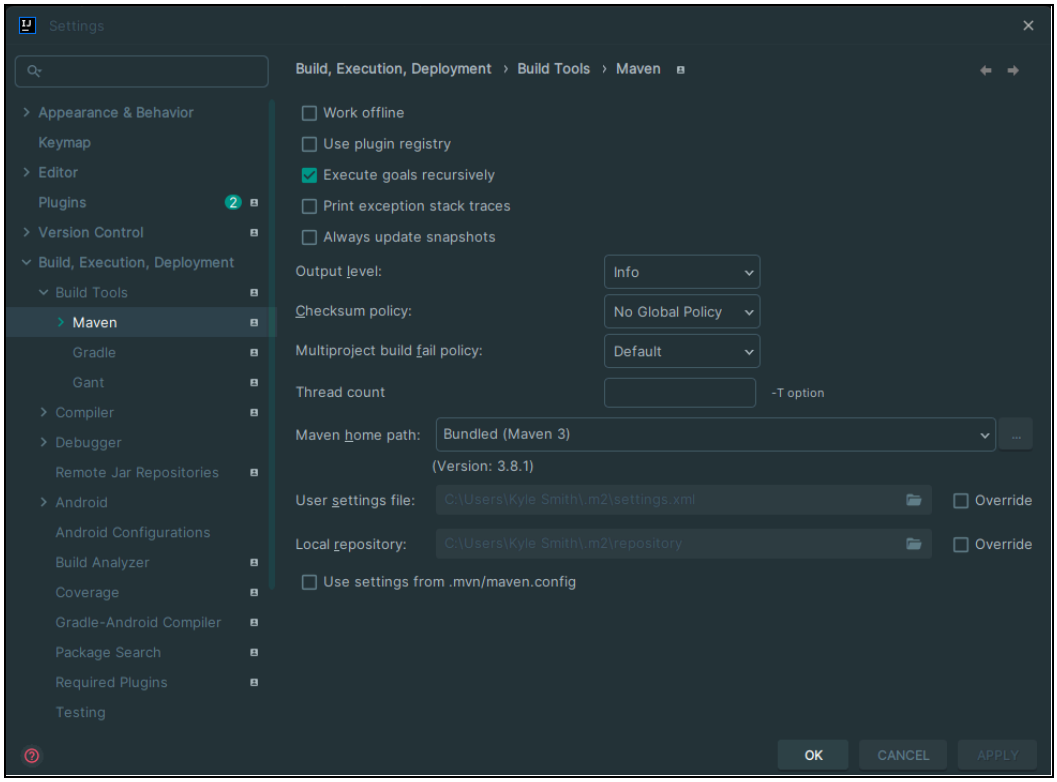
Getting started with unit testing will require you to have the JUnit plugin installed for IntelliJ. After you have Installed the plugin you will be ready to create new unit test classes and run them in IntelliJ. Further instructions for installing the JUnit plugin, setting up tests for your code, and running those tests are below.

IDE Setup for Unit Testing (IntelliJ JUnit Plugin)

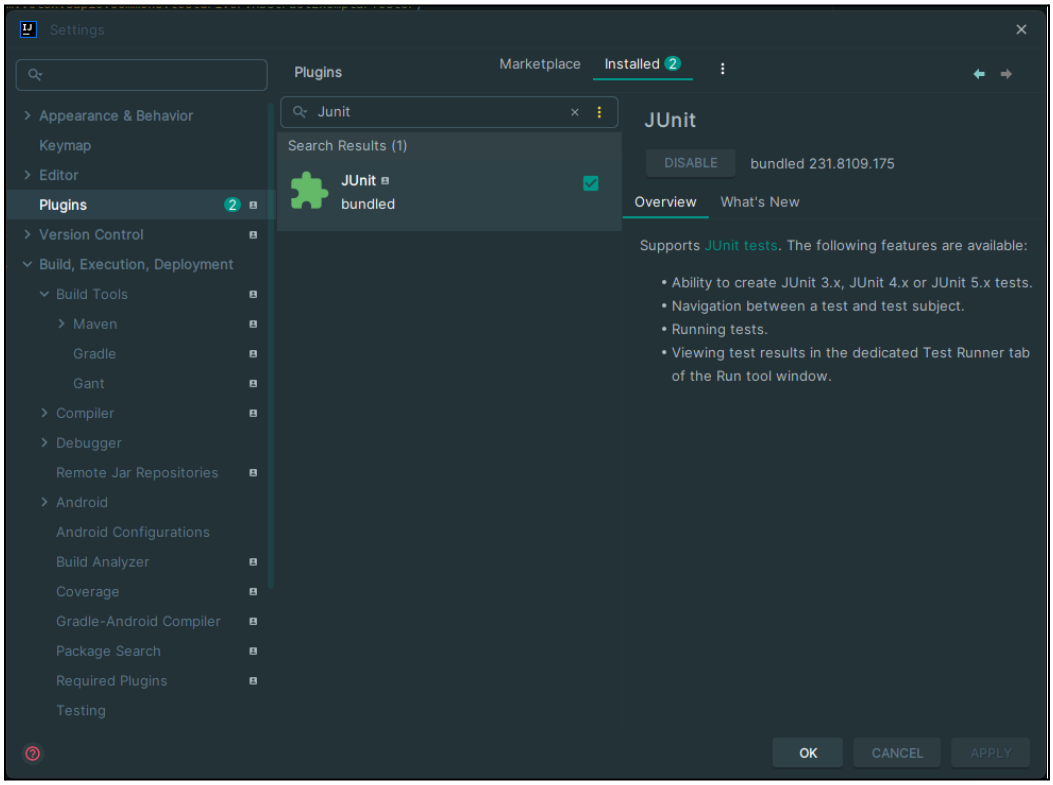
In order to install the Junit plugin you will first need to navigate to the settings menu by navigating to “File“ → “Settings“

1. In IntelliJ, navigate to "File" -> "Settings"

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**



2. Select "Plugins" from the left-hand menu



400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

3. By default, you should see your installed plugins. If "JUnit" is not listed or enabled:
    - Click the "Marketplace" tab
    - Search for "JUnit" and install the official plugin (often bundled)

Once the JUnit plugin is installed and enabled, your IntelliJ is ready for unit testing

Project Setup for Unit Testing

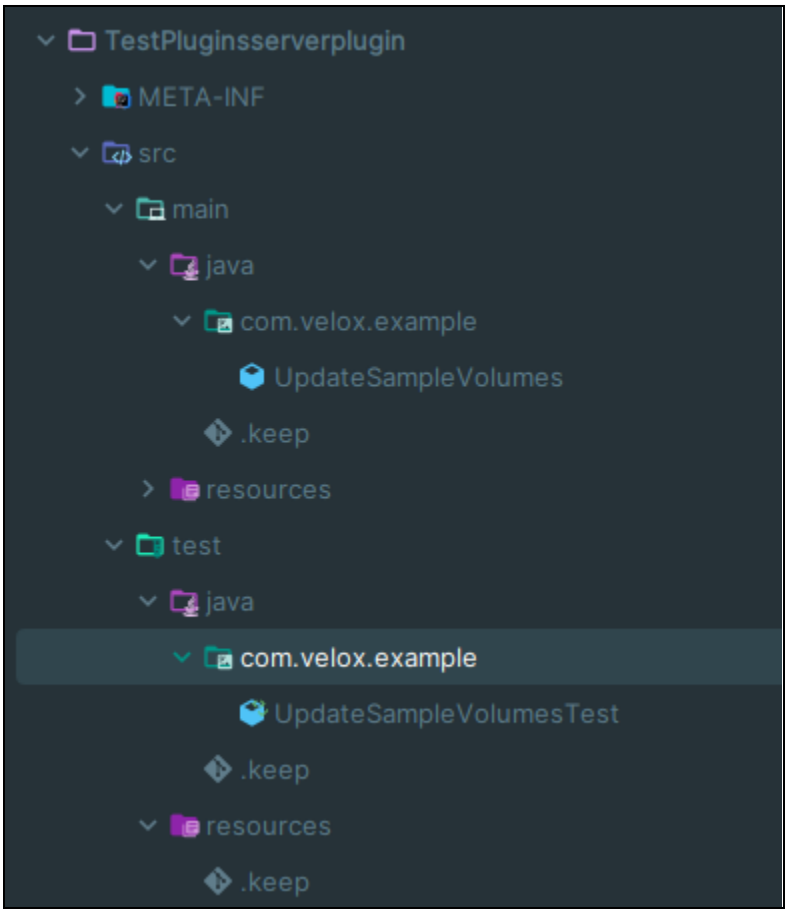
When a plugin package is created in a Sapio project, a test directory will be (or should be) created at the same level as the main directory.

1. To create a test for a particular class:
    - Replicate the folder structure of the class you are testing within the src/test/java directory
    - Create a new Java class with the name <ClassName>Test (e.g., if your class is UpdateSampleVolumes, the test class will be UpdateSampleVolumesTest)

    WARNING : The class name must end with “Test” or it will not be automatically retested on compile. The test must also be in src/test/java or it will not be triggered automatically on compile.

```
your-plugin-module/
├── pom.xml
└── src/
    ├── main/
    │   ├── java/
    │   │   └── com/velox/example/
    │   │       └── UpdateSampleVolumes.java
    │   └── resources/
    └── test/
        ├── java/
        │   └── com/velox/example/
        │       └── UpdateSampleVolumesTest.java
        └── resources/
            └── com/velox/example/  (for test-specific resources like synergy XML)
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**



Unit Test Class Setup (extending AbstractExemplarTester)

Your new test class *must* extend AbstractExemplarTester to utilize the mocked Sapio environment.

```
Java
// src/test/java/com/velox/example/UpdateSampleVolumesTest.java
package com.velox.example;

import com.velox.sapioutils.JUnit.AbstractExemplarTester; // Ensure correct import
import org.JUnit.jupiter.api.Test;
import static org.JUnit.jupiter.api.Assertions.*; // For assertions

public class UpdateSampleVolumesTest extends AbstractExemplarTester {

  // This method is automatically called before each @Test method
  @Override
  protected void additionalBeforeEach() throws Throwable {
    super.additionalBeforeEach(); // Good practice to call super
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```
        // Initialize common test data or load synergy files here (see next section)
        // Example: user.setEmailAddress("test@example.com");
    }

    @Test
    public void testSetVolume_PositiveCase() {
        // Test-specific setup
        // ...

        // Execute the method/plugin being tested
        // ...

        // Assert the expected outcome
        assertTrue(true, "Condition should be true"); // Replace with actual assertions
    }

    // Add more @Test methods for other scenarios
}
```

- **AbstractExemplarTester** provides access to mocked instances like dataMgmtServer, user, dataRecordManager, testMan, etc.
- Test methods should be public void and annotated with @Test
- Method names should clearly describe the case being tested

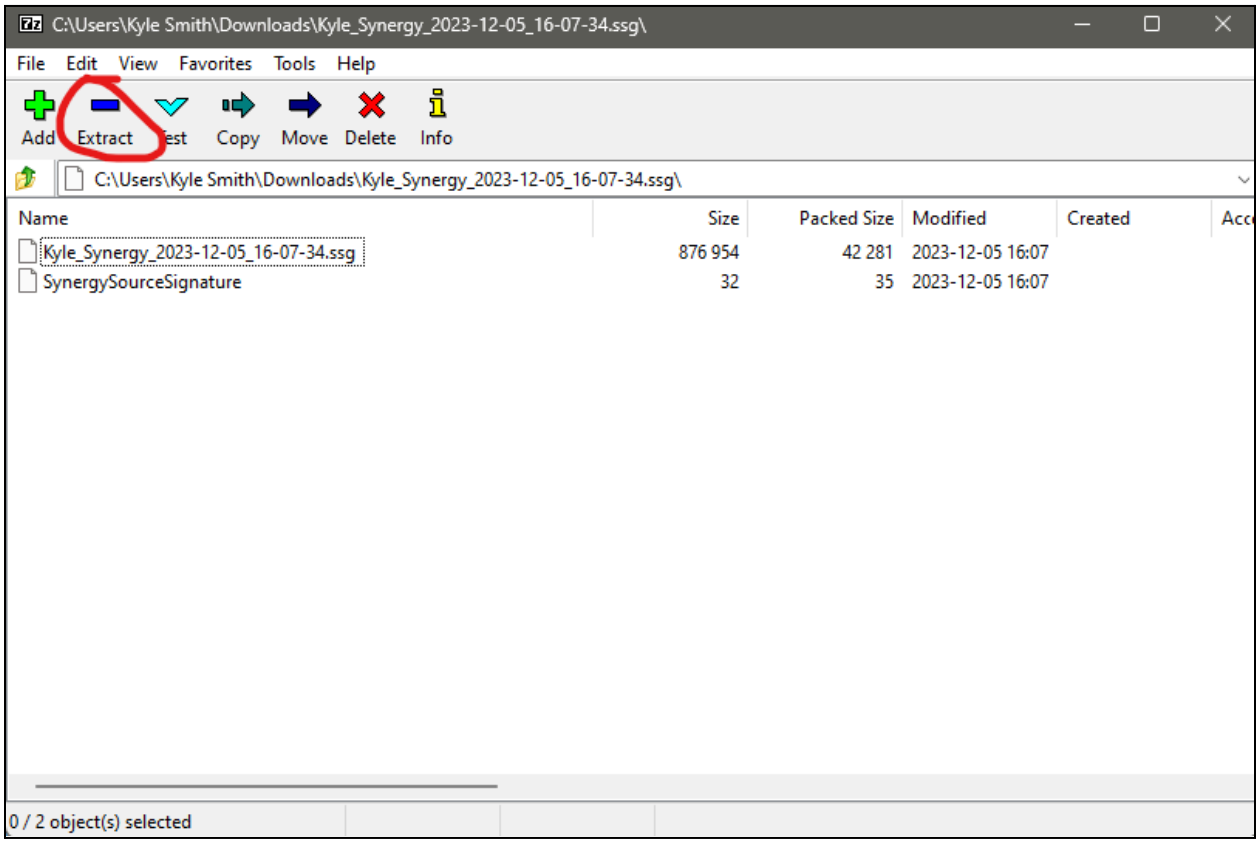
Mocking Sapio Unit Test Resources

Datatypes and Datafields (Synergy Files)

To work with specific Sapio Data Types and Data Fields in your tests, you need to load them from a synergy export.

1. **Export Synergy:** From your Sapio application, export a synergy file (.ssg) that contains the required datatypes and predefined fields.
2. **Extract XML:**
    - Open the .ssg file using 7-Zip or a comparable extraction tool
    - Extract the contents. You should find an XML file (often the same name as the .ssg file but with an .xml extension, or you might need to rename the extracted .ssg to .xml). Let's call it your_synergy_name.xml.

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**



3. **Place XML in Resources:** Copy your_synergy_name.xml into your test resources directory, typically mirroring the package structure (e.g., src/test/resources/com/velox/example/your_synergy_name.xml)

4. **Load Synergy in Test:** In order to load this synergy, you will need the following code

```
Java
var synergyFileName = "/<your_synergy_name>.xml";
    InputStream stream = getClass().getResourceAsStream(synergyFileName);
    byte[] data = IOUtils.toByteArray(stream);
    testMan.getDecryptedSynergyManager().addDataTypeDefinitions(data, true);
```

Use the additionalBeforeEach() method in your test class to load these definitions. Adding this code to your additionalBeforeEach method is a good way to ensure that your necessary datatypes will be in the test context for all of your test methods in a given test class.

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```java
@Override
protected void additionalBeforeEach() throws Throwable {
  super.additionalBeforeEach();
  user.setEmailAddress("admin@sapiosciences.com"); // Or any test user

  // Load Synergy datatypes
  String synergyFileName = "/com/velox/example/your_synergy_name.xml"; // Path relative to
      resources root
  java.io.InputStream stream = getClass().getResourceAsStream(synergyFileName);
  if (stream == null) {
    throw new RuntimeException("Cannot find synergy file: " + synergyFileName);
  }
  byte[] data = org.apache.commons.io.IOUtils.toByteArray(stream); // Requires commons-io
  testMan.getDecryptedSynergyManager().addDataTypeDefinitions(data, true); // 'true' to overwrite
}
```

Creating a Mock Experiment

In the case that you are testing something that requires an experiment, such as an entry toolbar or submission plugin, use the following code to create a mock experiment that you will have access to within your test.

```java
notebookExperiment = dataMgmtServer.getNotebookExperimentManager(user)
        .createNotebookExperiment("Test Experiment", null, user);
    testContextBuilder = ServerPluginContext.builder()
        .user(user)
        .notebookExperiment(notebookExperiment);
```

```java
// Inside a @Test method or additionalBeforeEach()
// ...
NotebookExperiment notebookExperiment = dataMgmtServer.getNotebookExperimentManager(user)
    .createNotebookExperiment("Test Experiment", null, user);

// If you need a ServerPluginContext for a plugin call:
ServerPluginContext testContext = ServerPluginContext.builder()
    .user(user)
    .notebookExperiment(notebookExperiment)
    // ... add other context items as needed
    .build();
// ...
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

Utilizing additionalBeforeEach()

This method is crucial for setting up common preconditions for *all* test methods in the class. This includes:

- Setting up the test user
- Loading synergy datatypes
- Creating commonly used mock records or experiments if they are needed by most tests

Writing and Running JUnit Tests

When running unit tests you have a few options for which test to run and how. You can run either…

- All test methods in all test classes within a particular directory
- All test methods within a particular test class
- Or a single test method

When running the test you can also specify whether to just run, run with debugger (to enable setting breakpoints), or to run with coverage (create data showing which parts of the code being tested are actually run by the test).

Basic Test Structure (already shown in Unit Test Class Setup)

- Extend AbstractExemplarTester
- Use @Test annotation for test methods
- Use JUnit Jupiter Assertions (e.g., assertEquals, assertTrue, assertNotNull)

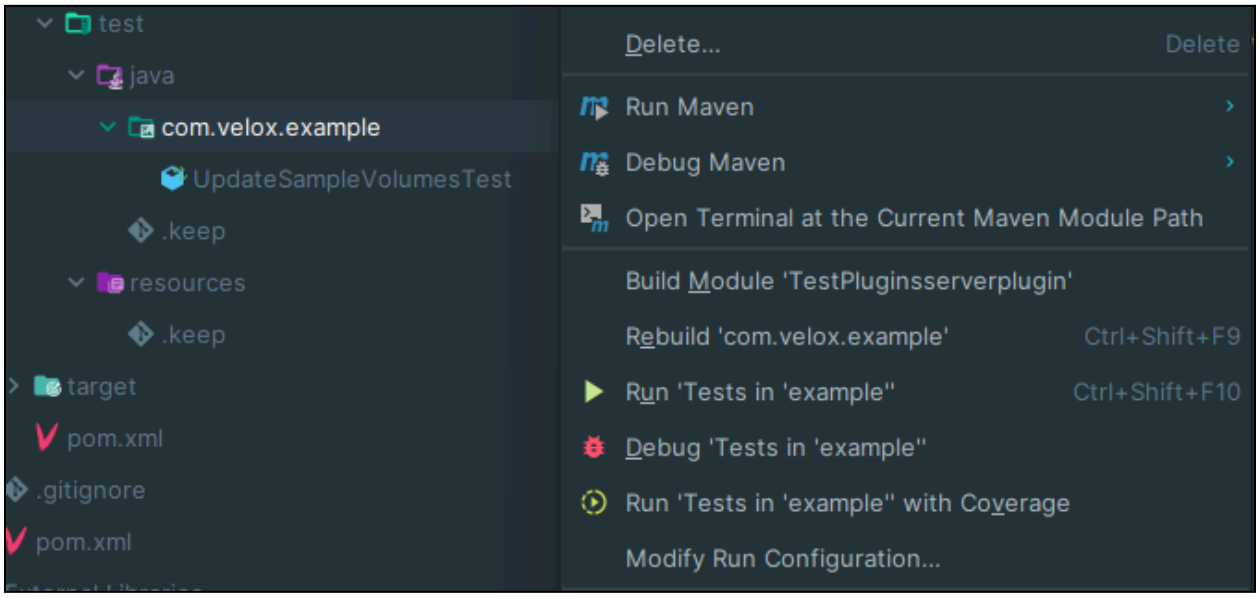
Running Tests from Project Panel, Class, or Method (IntelliJ)

From Project Panel (Directory or Class)

- In the Project panel, right-click on a test directory (e.g., com.velox.example) or a specific test class (e.g., UpdateSampleVolumesTest.java)

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

- Select "Run 'Tests in 'example'"" or "Run 'UpdateSampleVolumesTest'"
- You can also choose "Debug..." or "Run ... with Coverage".



From a Class or Method in Editor

In order to run either all tests within a test class or a particular test method within a class, you can right click the run button (play icon) on the line containing either the class or test method declaration that you would like to run and select how you would like to run it. In this example...

- Open the test class file
- Click the green "play" icon in the gutter next to the class declaration to run all tests in the class
- Click the green "play" icon next to a specific @Test method declaration to run only that method
- Select the desired run option (Run, Debug, Coverage).

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```
public class UpdateSampleVolumesTest extends AbstractExemplarTester {

    1 usage
    @Override
    protected void additionalBeforeEach() throws Throwable {
    }

    @Test
    public void testSetVolume(){
        // ToDo: Implement method to test setting sample volumes
        assertTrue( condition: true);
    }
}
```

Debugging JUnit Tests

1. Set breakpoints in your test code or the application code being tested
2. Run the test using the "Debug" option (as described above)
3. The execution will pause at your breakpoints, allowing you to inspect variables and step through the code

Viewing Test Results in IntelliJ

When you run tests, the "Run" panel will appear at the bottom of IntelliJ, showing:

- A tree view of executed tests and their status (pass/fail)
- Console output, including any System.out.println or logger messages
- For failed tests, stack traces and assertion failure details

```
Run    example in TestPluginsserverplugin
Tests passed: 1 of 1 test – 1 sec 679 ms
example (com.velox)                1 sec 679 ms
  UpdateSampleVolumesTest          1 sec 679 ms
    testSetVolume()                1 sec 679 ms
"C:\Program Files\Java\zulu18.32.13-ca-fx-jdk18.0.2.1-win_x64\bin\java.exe" ...
SLF4J: No SLF4J providers were found.
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#noProviders for further details.
SLF4J: Class path contains SLF4J bindings targeting slf4j-api versions prior to 1.8.
SLF4J: Ignoring binding found at [jar:file:/C:/Users/Kyle%20Smith/.m2/repository/org/apache/logging/log4j/log4j-slf4j-impl/2.17.2/log4j-slf4j-impl-2.17.2.jar!/org/slf4j/impl/St
SLF4J: See http://www.slf4j.org/codes.html#ignoredBindings for an explanation.
Test DataMgmtServer has been deleted.
Registered server-plugin: com.velox.plugin.VersionLogger
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

Measuring Test Coverage

Most IDEs (like IntelliJ) offer built-in test coverage tools

1. Run your tests using the "Run ... with Coverage" option
2. After execution, the IDE will:
    - Highlight lines of code in your source files:
        - **Green lines:** Code executed by tests
        - **Red lines:** Code not executed by tests
        - **Yellow lines (sometimes):** Partially covered code (e.g., a conditional where only one branch was taken).
    - Provide a coverage report, often showing percentages per package, class, method, and line.
3. Aim for high line coverage. Configure your run configuration (Edit Configurations -> select your test -> Code Coverage tab) to specify packages to include/exclude.

Integrating with Maven

The Maven build system automates the build process, including running unit tests.

- **Test Source Code:** src/test/java/
- **Test Resources:** src/test/resources/
- **Test Class Naming:** The maven-surefire-plugin (which runs unit tests) by default scans for classes matching patterns like /**/*Test.java, /**/Test*.java, or /**/*TestCase.java.

Dependencies: Test-specific dependencies (JUnit, Mockito, Sapio test utilities) should be declared with <scope>test</scope> in your pom.xml.

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```
Unset
    <dependency>
      <groupId>org.JUnit.jupiter</groupId>
      <artifactId>JUnit-jupiter-api</artifactId>
      <version>5.8.2</version>
      <scope>test</scope>
    </dependency>

    XML
```

- **Build Failure:** If any unit test fails when running mvn test or mvn install, Maven will mark the build as "BUILD FAILED."

Behavior-Driven Development (BDD) with Cucumber

BDD bridges the gap between technical implementation and business requirements by allowing tests to be written in a natural, human-readable language.

Why BDD?

BDD helps ensure all stakeholders have a common understanding of how the software should behave.

The BDD Workflow

1. **Outline Requirements:** Define specific story scenarios
2. **Write Feature Files:** Translate scenarios into Gherkin syntax in .feature files
3. **Implement Step Definitions:** Write Java code to execute the steps in feature files
4. **Write Application Code:** Develop code to make scenarios pass

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

Introducing Cucumber

Cucumber is a BDD tool using three main file types

1. **Feature Files (.feature):** Plain English scenarios
2. **Step Definition Files (.java):** Java code implementing steps
3. **Runner Files (.java):** JUnit classes to execute Cucumber features

Location of Cucumber Files

- **Feature files:** src/test/resources/com/yourorg/yourmodule/features/
- **Step Definitions & Runner files:** src/test/java/com/yourorg/yourmodule/features/ (same package as features is common)

IDE Plugin for Cucumber

Install "Cucumber for Java" plugin in IntelliJ for syntax highlighting, navigation, etc.

Crafting Feature Files (.feature)

Use Gherkin: Feature, Scenario, Given, When, Then, And, But. Tags (e.g., @SmokeTest) can be organized.

```
Unset
    #
    src/test/resources/com/velox/example/features/sample_update.feature
    @SampleManagement
    Feature: Sample Volume Update
     As a lab technician,
     I want to update sample volumes based on an entry option,
     So that sample records reflect accurate quantities.

     Scenario: Update sample volume based on "Full" option
      Given a sample "S001" exists with an initial volume of 10.0
      And the entry option for volume update is set to "Full"
      When the sample volume update plugin is executed for "S001"
      Then the volume of sample "S001" should be 100.0

    GHERKIN
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

Implementing Step Definitions (.java) - Leveraging AbstractExemplarTester

Step definition classes should also extend AbstractExemplarTester if they need to interact with the mocked Sapio environment.

```Java
// src/test/java/com/velox/example/features/SampleUpdateSteps.java
package com.velox.example.features;

import com.velox.sapioutils.JUnit.AbstractExemplarTester;
import io.cucumber.java.en.Given;
import io.cucumber.java.en.When;
import io.cucumber.java.en.Then;
import static org.JUnit.jupiter.api.Assertions.*;
// Import necessary Sapio classes (DataRecord, Plugin interfaces etc.)

public class SampleUpdateSteps extends AbstractExemplarTester {

  private String currentSampleId;
  private String entryOptionValue;
  // You might store DataRecord objects or other state here

  // Cucumber's @Before can be used like additionalBeforeEach for BDD scenario setup
  @io.cucumber.java.Before
  public void setUpScenario() throws Throwable {
    additionalBeforeEach(); // Calls the setup from AbstractExemplarTester
    // Load common synergy files if not already in parent's additionalBeforeEach
  }

  @Given("a sample {string} exists with an initial volume of {double}")
  public void a_sample_exists_with_initial_volume(String sampleId, Double initialVolume) {
    this.currentSampleId = sampleId;
    // Use dataMgmtServer.getDataRecordManager(user).addDataRecord("Sample", user);
    // Set sampleId and initialVolume fields on the mock DataRecord
    // ...
  }

  @Given("the entry option for volume update is set to {string}")
  public void the_entry_option_for_volume_update_is_set_to(String optionValue) {
    this.entryOptionValue = optionValue;
    // Mock setting this option, perhaps on a parent record or context
    // ...
  }

  @When("the sample volume update plugin is executed for {string}")
  public void the_sample_volume_update_plugin_is_executed_for(String sampleId) {
    // Find the mock DataRecord for sampleId
    // Call your actual UpdateSampleVolumes plugin/logic here
    // Example:
    // UpdateSampleVolumesPlugin plugin = new UpdateSampleVolumesPlugin();
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```
        // ServerPluginContext ctx = ServerPluginContext.builder().user(user)...build(); // build context
        // plugin.execute(ctx, recordList);
        // ...
    }

    @Then("the volume of sample {string} should be {double}")
    public void the_volume_of_sample_should_be(String sampleId, Double expectedVolume) {
        // Retrieve the mock DataRecord for sampleId
        // DataRecord sample = ... ;
        // assertEquals(expectedVolume, sample.getDoubleVal("Volume", user));
        // ...
    }
}
```

- Use Cucumber's @Before (from io.cucumber.java.Before) for scenario-specific setup, and call additionalBeforeEach() if you need the base setup.

Using a Test Described in a Table for Batch Conditions

Use the scenario outline, like the one below, to describe all assumptions and test parameters for a test that uses a table. Then, use a table of examples to enter all variables in the scenario outline. This is useful when you have tests that are more of the same for multiple conditions.

```
Java
#[FR-33530] - Create a common cucumber steps class for common storage
    management functions.
@StoreageManagement
Feature: Test Storage Management Common Cucumber Steps
  This feature is used to test the common Cucumber steps for the Storage
    Management module.

  Background: A clean server with NGS synergy, with an Exemplar
    Configuration record created
     Given clean DataMgmtServer with NGS baseline synergy
     Then type Sample exists
     Then type StorageUnit exists
     Then type StorageConfiguration exists
     Then type StorageHierarchyConfig exists
     Given a new data record of type ExemplarConfig, call it Config
     And Store Changes
     Then data record Config has type ExemplarConfig

  Scenario Outline: Populating the Storage Unit Types Pick List
     Given Storage Unit Types: <types>
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```
  Then Storage Unit Types are set: <types>

  Examples:
    | types          |
    | Box            |
    | Shelf,Rack,Box |

Scenario Outline: Populating the Storage Unit Types Hierarchy
  Given Storage Unit Types: <types>
  Given Storage Hierarchy: <hierarchy>
  Then Storage Hierarchy is set: <hierarchy>

  Examples:
    | types          | hierarchy                          |
    | Box            | Box                                |
    | Shelf,Rack,Box | Shelf\|Rack\|Box                   |
    | Shelf,Rack,Box | Shelf\|Rack\|Box:::Rack\|Box:::Box |

Scenario Outline: Populating the Storage Items
  Given Storage Unit Types: <types>
  Given Storage Hierarchy: <hierarchy>
  Given Storable Item <datatype1> with field <datafield1> and storage unit
 types: <types>
  Then StorageConfiguration Storable Item <datatype1> is set with field
 <datafield1> and storage unit types: <types>
  Given Storable Item <datatype2> with field <datafield2> and storage unit
 types: <types>
  Then StorageConfiguration Storable Item <datatype2> is set with field
 <datafield2> and storage unit types: <types>

  Examples:
    | types          | hierarchy                          | datatype1 |
 datafield1 | datatype2 | datafield2 |
    | Shelf,Rack,Box | Shelf\|Rack\|Box                   | Sample    |
 SampleId   | Request   | RequestId  |
    | Shelf,Rack,Box | Shelf\|Rack\|Box:::Rack\|Box:::Box | Sample    |
 SampleId   | Request   | RequestId  |

Scenario Outline: Populating the Storage Units
  Given Storage Unit Types: <types>
  Given Storage Hierarchy: <hierarchy>
  Given Storable Item <datatype> with field <datafield> and storage unit
 types: <types>
  Given Storage Unit Hierarchy: <unithierarchy>
  Then Storage Unit Hierarchy is set: <unithierarchy>

  Examples:
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```
    | types          | hierarchy                          | datatype |
  datafield | unithierarchy
  |
    | Rack,Box       | Rack\|Box                          | Sample   |
  SampleId  |
  StorageUnitType=Box\|UnitRows=8\|UnitColumns=8\|NumberToCreate=1
  |
    | Shelf,Rack,Box | Shelf\|Rack\|Box                   | Sample   |
  SampleId  |
  StorageUnitType=Rack\|UnitRows=2\|UnitColumns=2\|NumberToCreate=4:::Stora
  geUnitType=Box\|UnitRows=8\|UnitColumns=8\|NumberToCreate=4
  |
    | Shelf,Rack,Box | Shelf\|Rack\|Box:::Rack\|Box:::Box | Sample   |
  SampleId  |
  StorageUnitType=Shelf\|UnitRows=2\|UnitColumns=2\|NumberToCreate=4:::Stor
  ageUnitType=Rack\|UnitRows=2\|UnitColumns=2\|NumberToCreate=4:::StorageUn
  itType=Box\|UnitRows=8\|UnitColumns=8\|NumberToCreate=4 |

Scenario: Create default storage hierarchy and items
  Given Create default storage hierarchy and items and initial storage
  units
  Then Storage Unit Types are set: Freezer,Shelf,Rack,Box
  Then Storage Hierarchy is set: Freezer|Shelf|Rack|Box
  Then StorageConfiguration Storable Item Sample is set with field
  SampleId and storage unit types: Box
  Then Storage Unit Hierarchy is set:
  StorageUnitType=Freezer|UnitRows=2|UnitColumns=2|NumberToCreate=1:::Stora
  geUnitType=Shelf|UnitRows=2|UnitColumns=2|NumberToCreate=4:::StorageUnitT
  ype=Rack|UnitRows=2|UnitColumns=2|NumberToCreate=4:::StorageUnitType=Box|
  UnitRows=8|UnitColumns=8|NumberToCreate=4

Scenario Outline: Assign storage items to storage units
  Given Create default storage hierarchy and items and initial storage
  units
  Given a new data record of type <datatype>, call it Test
  And Store Changes
  Then data record Test has type <datatype>
  Given Assign storable item Test to <unit> position <pos>
  Then storable item Test has been stored

  Examples:
    | datatype | unit | pos |
    | Sample   | Box1 | A1  |
    | Sample   | Box3 | C4  |

Scenario Outline: Assign storage items to storage units
  Given Create default storage hierarchy and items and initial storage
  units
  Given a new data record of type <datatype>, call it Test
```


400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

```
    And Store Changes
    Then data record Test has type <datatype>
    Then storable item Test is not in storage
    Given Assign storable item Test to <unit> position <pos>
    Then storable item Test has been stored
    Given Remove from storage Test
    Then storable item Test is not in storage

    Examples:
      | datatype | unit | pos |
      | Sample   | Box1 | A1  |
      | Sample   | Box3 | C4  |
```

Setting Up the Cucumber Test Runner (.java)

```java
// src/test/java/com/velox/example/features/RunCucumberTest.java
package com.velox.example.features;

import io.cucumber.JUnit.Cucumber;
import io.cucumber.JUnit.CucumberOptions;
import org.JUnit.runner.RunWith; // From JUnit 4 for Cucumber runner

@RunWith(Cucumber.class) // Uses JUnit 4 runner for Cucumber
@CucumberOptions(
  features = "src/test/resources/com/velox/example/features",
  glue = "com.velox.example.features", // Package with step definitions
  plugin = {"pretty", "json:target/cucumber-reports/cucumber.json"},
  monochrome = true
)
public class RunCucumberTest {
}
```

Note : Cucumber 7+ can use JUnit Platform (Jupiter and JUnit 5) Engine. If using older Cucumber, @RunWith(Cucumber.class) is standard.

Debugging Cucumber Tests

- **IDE (with plugin):** Right-click on a "Feature" or "Scenario" line, or a specific Gherkin step, and select "Debug"
- **Via Runner:** Set breakpoints in step definitions and debug RunCucumberTest.java as a JUnit test

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

Automated Reporting and CI/CD

Generating Reports with Maven

When mvn test runs, if Cucumber tests are executed via the runner, the json:target/cucumber-reports/cucumber.json file will be generated.

Visualizing Results in Jenkins (or other CI servers)

- Install a "Cucumber reports" plugin in Jenkins
- Configure your Jenkins job to publish these reports, pointing to the path of the generated JSON file (e.g., target/cucumber-reports/cucumber.json)
- This provides an interactive HTML report with pass/fail statistics, breakdowns by tags, scenario details, and trends

Best Practices & Tricks of the Trade

- **Synergy Files:** Ensure ALL Data Types and Data Fields you intend to test *and their dependencies* are present in the synergy XML loaded in additionalBeforeEach()
- **Record Creation:** All records (Samples, Experiments, Process Steps, etc.) needed for a test must be explicitly created in additionalBeforeEach() or within the @Test method. They are not imported by the synergy XML. This includes assigned processes and their child workflows.
- **Test One Thing:** Each @Test method should ideally test a single logical case or branch of functionality. Name tests clearly.
- **Cover All Branches:** Test all possible logic branches in your plugin/method, including expected error conditions. Write tests that provide input to produce errors and assert that the correct error (or state) occurs.
- **Unit Focus (for AbstractExemplarTester):**
    - Focus on the single unit (plugin/service). Do not assume other plugins or unrelated system invocations exist or will trigger.
    - If on-save behavior needs testing, create separate unit tests: one for the plugin's direct input/output, and another specifically for the on-save trigger's input/output.

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**

- **AbstractExemplarTester Limitations:**
    - Parallel test runs against multiple distinct database instances might not be fully supported by the base mock environment
    - Complex SQL-specific calls (e.g., via a SqlBuilder) might have limited support. Basic DataRecordManager queries are usually fine
    - Support for deprecated methods or poorly designed components may be absent
    - The test driver is a utility; continuous improvement is expected
- **Reusability (BDD):** Well-crafted step definitions can be reused across many scenarios, making feature file creation the primary effort for new tests

Conclusion

Implementing this unit testing framework combines the precision of JUnit, the environmental control of AbstractExemplarTester, the automation of Maven, and the clarity of BDD with Cucumber. This approach offers:

- Development of better products, cheaper, and faster
- Capture of requirements directly in testable, human-readable formats
- Easy-to-read specifications for validators and stakeholders
- Automated testing and report generation, integrating with CI/CD
- More effective delegation of functional code development

This structured approach fosters a culture of quality and collaboration, leading to more robust and reliable software on the Sapio platform.

400 E. Pratt Street Suite 800
Baltimore, MD 21202
(410) 800-4620
This document is **RESTRICTED**